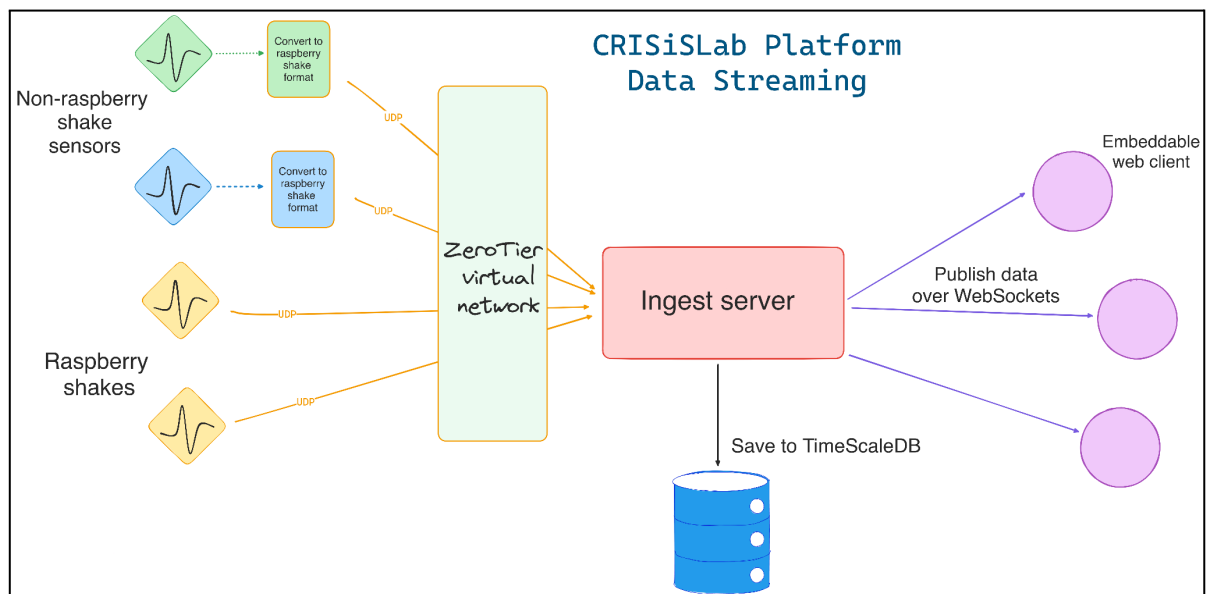


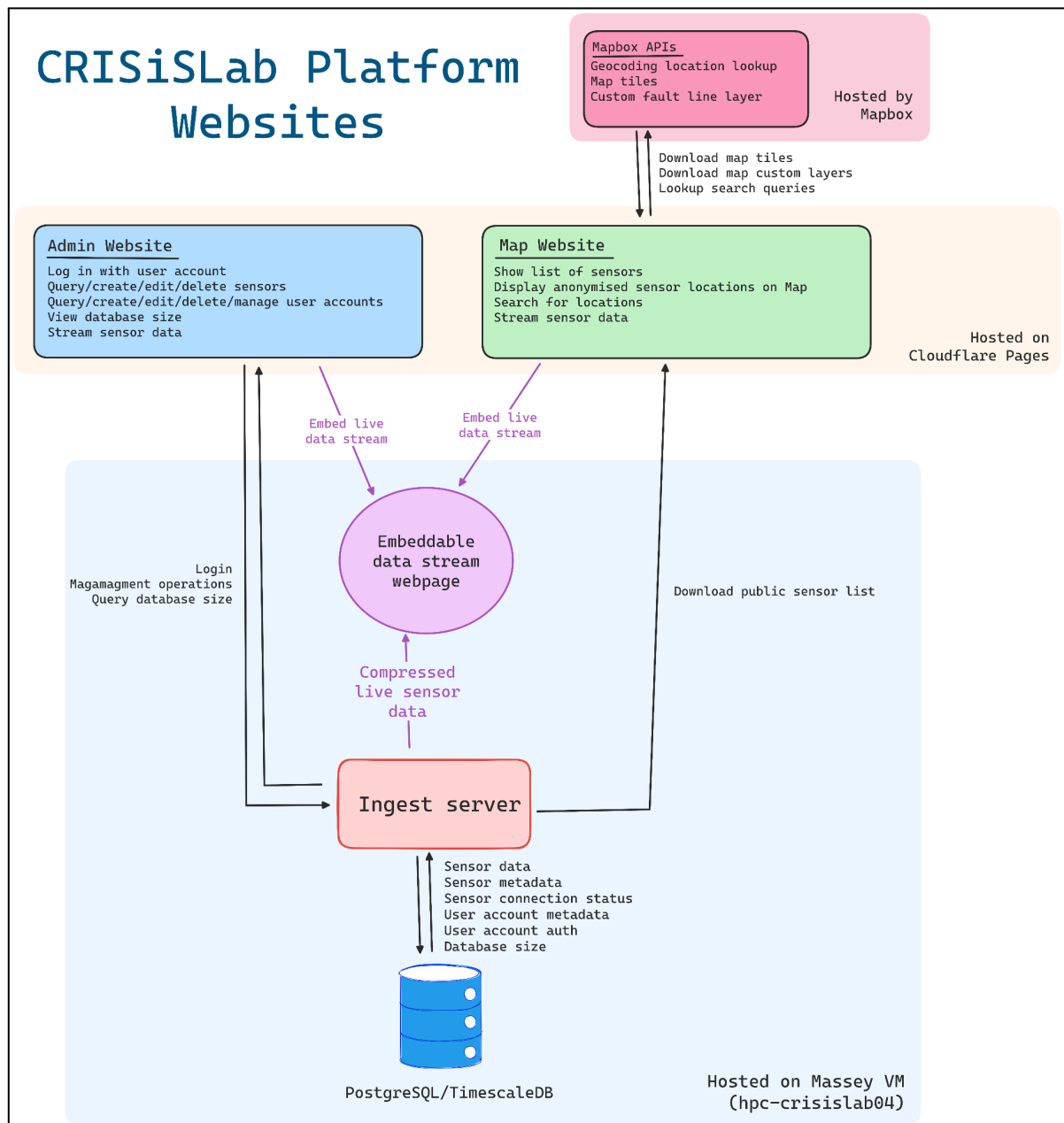
CRISiSLab Platform Documentation

Zade Viggers, June 2024

This is a technical overview of how we at CRISiSLab run our seismic data (and occasionally other data type) ingest, storage, and streaming platform.

Architecture diagrams





Index of components

- Raspberry Shake sensors are small sensors built on top of a Raspberry Pi that can stream data via UDP. We use their data format as our standard for all sensors. Most of our Raspberry Shakes are 4D models.

The different models we use are

- Raspberry Shake 4D
- Raspberry Shake and Boom
- CSI EQResponder
- PAlert Plus
- CRISiSLab/Mithira in-house sensor
- GSD SeismicDataSensor

- Non Raspberry Shake sensors: We have a few sensors from manufacturers other than Raspberry Shake. These always have a Raspberry Pi attached to them that translates the sensor's data into the Raspberry Shake data format, before sending it to the server over UDP. [Here's the code for those translators](#).
- ZeroTier virtual network: Zerotier is software that can be installed on any computer. It creates a virtual network, so all devices on the same network can act as if they were all attached to the same router, even if they're on opposite sides of the world. All of our Raspberry Shakes, data translation Raspberry Pis, and the server are on the network. It requires approval to join, so removes the need for authentication on the data streams from sensors.
- [Ingest server](#): also known as 'ingest-deno' this server is the heart of our system. It runs as a single process on the VM hpc-crisislab04. It receives data from all of the sensors, and then saves it into the database, and streams it to any web clients. It tracks which sensors are currently sending data (AKA 'online') and saves that in the database. The server hosts an API with an endpoint for retrieving a list of sensors with anonymised locations. If a user can authenticate with the server, it has endpoints that allow modifying sensor metadata, retrieving non-anonymised sensor locations, managing user accounts, and getting the size history of the database. The server also hosts an embeddable web client to stream live data. It is written in Typescript and runs on the Deno runtime.
- TimescaleDB/PostgreSQL: The database we use to store sensor data. TimescaleDB acts as an extension for PostgreSQL, adding support for special 'hypertables' that store time-series data very efficiently. This database also stores sensor metadata, user account data, and size history for itself. It runs as a separate process from the main server. It is also sometimes used by other programs to store data.
- Websockets: A web technology that makes it simple to stream real-time data to a website. Ours have some special extra compression applied (called msgpack) to reduce bandwidth usage.
- [Embeddable web client](#): This is a simple single-page application that is statically hosted by the server. It connects to the server's websockets and graphs the sensor data in real-time using our graphing library TimeLine. It's written in TypeScript but doesn't use a UI library. You can view it online at <https://crisislab-data.massey.ac.nz/>.
- Mapbox: Mapbox is a map rendering library and hosting service. We use their library to render the maps for the map website. We use their Geocoding API to enable place-based search on the map website. We use their tile hosting service for loading map tiles (that's the actual map part of a map), and their layer service for hosting our custom fault lines layer.
- [Admin website](#): A web app that users can log into to manage sensors and user accounts, and view the size history for the database. It also has a few pages that work without logging in, but those display the same data that the public map website displays. It also embeds the data streaming web client for viewing graphs and uses our graphing library [TimeLine](#) to show the database size history. It's written in TypeScript and uses React and MUI. It is statically hosted on Cloudflare pages. You can view it online at <https://admin.crisislab.org.nz/>.
- [Map website](#): A web app that shows all our sensors on a map. If a user clicks on a sensor on the map, it shows them that sensor's live data in the sidebar by embedding the data streaming web client. It has a search function that supports quick jumping to sensor IDs using #, quick jumping to station IDs, and looking up sensors by location

using Mapox Geocoding. It also has options to show the locations of fault lines, and the locations of Geonet sensors. It uses Mapbox to render the map. It's written in TypeScript and uses React and MUI. It is statically hosted on Cloudflare pages. You can view it online at <https://shakemap.crisislab.org.nz/>.

Adding a new sensor

Please see the document “How to set up a CRISiSLab sensor” (which should be co-located with this document. If you are a CRISiSLab member or partner, you may request access to this live internal version: [How to set up a sensor \(CRISiSLab internal\)](#))

Running the system

1. Set up a Zerotier network
2. Set up the [Ingest server](#) on a VM and connect it to the Zerotier network. The repository's readme file provides setup instructions. Make sure it has a public IP or domain.
3. Deploy the [Admin website](#) and [Map website](#) to your choice of a static host (we normally use Cloudflare Pages), and update the API endpoints and data streaming URL constants in the code to point to the new server you set up.
4. Set up some sensors and connect them to your network. Here are instructions on [how to set up a sensor](#). You will need to swap out our network ID for your own network ID, and our server's Zerotier IP for your won server's Zerotier IP.
5. Done!